



Phase 1 - Tokyo Foundation Code (Sanitized)

This phase establishes the Tokyo Transit Gateway hub. **Tokyo must be deployed first** because São Paulo's peering references Tokyo's TGW.

File 1: 02-providers.tf (ADD São Paulo alias)

What it does: Adds São Paulo provider so Tokyo can create the peering attachment.

ADD this block to your existing providers file:

```
# ADD TO EXISTING
provider "aws" {
  alias   = "saopaulo"
  region = "sa-east-1"
}
```



Your existing providers stay unchanged: `aws` (default, ap-northeast-1) and `aws.us _east_1` for CloudFront.

File 2: 03-variables.tf (ADD Lab 3 variables)

ADD these variables to your existing variables file:

```
#####
# LAB 3 VARIABLES - ADD TO EXISTING FILE
#####
```

```

variable "tokyo_vpc_cidr" {
    description = "Tokyo VPC CIDR block (your Lab 2 VPC)"
    type        = string
    default     = "10.0.0.0/16" # Your Tokyo VPC
}

variable "saopaulo_vpc_cidr" {
    description = "São Paulo VPC CIDR block (must not overlap)"
    type        = string
    default     = "10.1.0.0/16" # Your São Paulo VPC
}

variable "tokyo_private_subnet_cidrs" {
    description = "Tokyo private subnet CIDRs"
    type        = list(string)
    default     = ["10.0.11.0/24", "10.0.12.0/24"]
}

variable "tokyo_public_subnet_cidrs" {
    description = "Tokyo public subnet CIDRs (for NAT Gateway)"
    type        = list(string)
    default     = ["10.0.1.0/24", "10.0.2.0/24"]
}

variable "saopaulo_private_subnet_cidrs" {
    description = "São Paulo private subnet CIDRs"
    type        = list(string)
    default     = ["10.1.11.0/24", "10.1.12.0/24"]
}

variable "saopaulo_public_subnet_cidrs" {
    description = "São Paulo public subnet CIDRs (for NAT Gateway)"
    type        = list(string)
    default     = ["10.1.1.0/24", "10.1.2.0/24"]
}

```

```

}

variable "saopaulo_tgw_id" {
  description = "São Paulo TGW ID (populated after Phase 2)"
  type        = string
  default     = ""
}

variable "enable_tgw" {
  description = "Enable Transit Gateway for Lab 3"
  type        = bool
  default     = true
}

```

File 3: 04-3a-tokyo-tgw.tf (NEW FILE)

What it does: Creates the Tokyo Transit Gateway hub and attaches it to your existing `helga_vpclab2a`.



Why Private Subnets? The TGW attachment uses private subnets because:

- Cross-region traffic to RDS is private-to-private
- No internet exposure needed for TGW routes
- Public subnets are still used for NAT Gateway placement

```

#####
# LAB 3A: TOKYO TRANSIT GATEWAY (HUB)
# Shinjuku Station - Data corridor hub
# Builds on: helga_vpclab2a, helga_private_subnets
#####

resource "aws_ec2_transit_gateway" "shinjuku_tgw01" {

```

```

count          = var.enable_tgw ? 1 : 0
description = "shinjuku-tgw01 (Tokyo hub for APPI-compliant
architecture)"

amazon_side_asn          = 64512
auto_accept_shared_attachments = "disable"
default_route_table_association = "enable"
default_route_table_propagation = "enable"
dns_support              = "enable"
vpn_ecmp_support        = "enable"

tags = {
  Name     = "shinjuku-tgw01"
  Role     = "Hub"
  Project  = var.project_name
}
}

```

```

#####
# TOKYO VPC ATTACHMENT TO TGW
# Connects to your existing helga_vpclab2a
# Attaches to PRIVATE subnets only
#####

```

```

resource "aws_ec2_transit_gateway_vpc_attachment" "shinjuku_a
ttach_tokyo_vpc01" {
  count          = var.enable_tgw ? 1 : 0
  transit_gateway_id = aws_ec2_transit_gateway.shinjuku_tgw01
[0].id

```

```

# References YOUR Lab 2 VPC
vpc_id = aws_vpc.helga_vpclab2a.id

# Attach to PRIVATE subnets only - TGW traffic stays intern
al
subnet_ids = [

```

```

    aws_subnet.helga_private_subnets[0].id,
    aws_subnet.helga_private_subnets[1].id
]

dns_support                                = "enable"
transit_gateway_default_route_table_association = true
transit_gateway_default_route_table_propagation = true

tags = {
    Name      = "shinjuku-attach-tokyo-vpc01"
    Project   = var.project_name
}
}

```

File 4: **05-outputs.tf** (ADD Lab 3 outputs)

ADD these outputs to your existing outputs file:

```

#####
# LAB 3 OUTPUTS - ADD TO EXISTING FILE
#####

output "tokyo_tgw_id" {
    description = "Tokyo Transit Gateway ID (for São Paulo peer
ing)"
    value      = var.enable_tgw ? aws_ec2_transit_gateway.shin
juku_tgw01[0].id : null
}

output "tokyo_tgw_arn" {
    description = "Tokyo Transit Gateway ARN"
    value      = var.enable_tgw ? aws_ec2_transit_gateway.shin
juku_tgw01[0].arn : null
}

```

```
output "tokyo_vpc_cidr_block" {
  description = "Tokyo VPC CIDR (for São Paulo routing)"
  value       = aws_vpc.helga_vpclab2a.cidr_block
}

output "tokyo_rds_endpoint" {
  description = "Tokyo RDS endpoint (for São Paulo app config)"
  value       = aws_db_instance.helga_rds01.endpoint
  sensitive   = true
}
```

Deployment Commands (Phase 1)

```
cd ~/lab3/tokyo

# Initialize (picks up new provider)
terraform init

# Validate
terraform validate

# Plan (should show TGW + VPC attachment)
terraform plan -out=phase1.tfplan

# Apply
terraform apply phase1.tfplan

# Capture TGW ID for Phase 2
terraform output tokyo_tgw_id
# Save this value - you'll need it for São Paulo
```

Verification Checklist (Phase 1)

- ☐ Tokyo TGW created → `tgw-XXXXXXXXXXXXXXXXXX`
- ☐ TGW attachment `shinjuku-attach-tokyo-vpclab3` is `available`
- ☐ Attachment uses private subnets (2 subnets attached)
- ☐ `terraform output tokyo_tgw_id` returns valid ID
- ☐ No changes to existing Lab 2 resources (CloudFront, ALB, RDS)

```
# Verify TGW created (list all TGWs in Tokyo)
aws ec2 describe-transit-gateways --region ap-northeast-1

# Verify TGW VPC attachments (list all attachments in Tokyo)
aws ec2 describe-transit-gateway-vpc-attachments --region ap-northeast-1
```



Phase 1 Complete: Proceed to Phase 2 to create São Paulo spoke.